

CMS and LIX UR5e robot plan

Justification

The goal of this robotic development initiative is to not only develop a generic robotic platform that is beamline and robot agnostic for standard sample handling, but to also expand the role of automation at the beamline. Instead of acting as a fixed-purpose mechanical arm, the next-generation beamline robot will evolve into a flexible operational tool capable of performing a wide range of tasks traditionally done by humans. This includes handling diverse experimental components, responding to the dynamic beamline environment, and ultimately supporting autonomous experimental workflows.

Importantly, the availability of a flexible, robotic platform is no longer optional for CMS. Several current and planned experiments depend on robotic handling due to safety constraints, environmental interlocks, precision requirements, and the need for reproducible, multi-step workflows. Without robotic automation, certain experiments cannot be performed at all, while others would be significantly constrained in throughput and reliability. The UR5e robot is critical for enabling the next phase of CMS scientific capabilities.

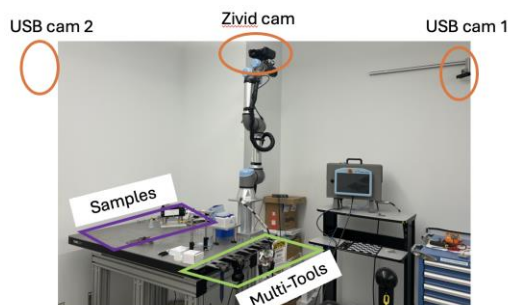
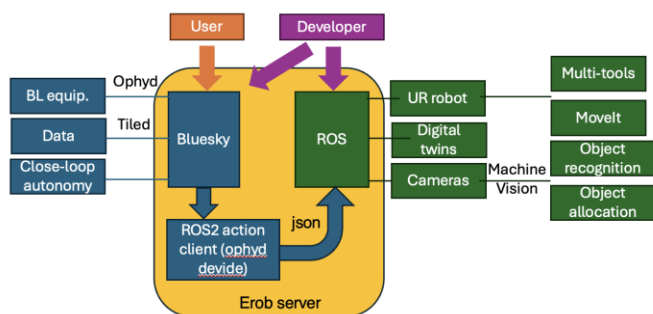
For LIX, the robot is an indispensable part of beamline operations. However, the current robot has limitations which make it difficult to implement new functionalities, especially when we began expanding beyond solution scattering experiments. The new robot will allow for different experimental configurations without requiring expert staff to tune the robot.

Developing this robotic platform will unlock new capabilities, including:

- i. performing experiments that require robotic handling due to environmental or interlock constraints.
- ii. operating multiple tools for in-situ and operando studies under autonomous control.

Goals:

1. Safe operation with high reliability and reproducibility.
2. Flexibility to handle multiple tools and perform complex, multi-step tasks.
3. Operational protocols developed for adoption across other beamlines.



Left: architecture of robotic operation with bluesky. Right: critical hardware components of the UR5e robot on a modular table.

Technical Approaches

The collaborative robot (cobot) platform enables safe human–robot interaction with extensive safety features. Universal Robots (UR) systems have already been demonstrated in operation at NSLS-II beamlines, including integration within the Bluesky data-acquisition environment. Building on these successful efforts, we will adapt and extend the UR-based robotics framework as a critical part to enable ‘Lab-at-Beamline’ (LaB). This creates a timely opportunity for CMS to partner with CFN and collaborate with LIX from CBMS, and DSSI to develop a shared set of robotic tools and workflows that can ultimately be deployed across multiple beamlines at NSLS-II.

Critical components include:

1. UR5e with ROS2 integration
 - 6-axis robot with built-in force/torque sensing (cobot)
 - Community-supported middleware for robotic applications
 - Compatible with standard robotics tools such as MoveIt2 and multi-modal camera systems
 - Previously demonstrated to operate within the Bluesky framework at NSLS-II
2. Optical cameras.
 - Real-time monitoring and scene reconstruction for safe operation
 - Points of cloud generation for object recognition and localization
 - 2x stereo camera for mapping large space (2x2x2m) and 1x camera mounted on robot arm for fine resolutions (<1mm)
3. Multiple sample handlers
 - Finger gripper
 - Vacuum-based holder
 - Pipettor holder for solution processing (custom-developed)
4. Bluesky access through virtual machine (TODO)
 - Portable control for beamline access
5. GPU resources (TODO)
 - Real-time point-cloud analysis Points of cloud monitoring
 - IsaacSim simulation

Computing Requirements: Ubuntu and GPU Dependency

The development and operation of the UR5e robotic system require a dedicated Ubuntu-based computer equipped with an NVIDIA GPU. This requirement is driven by fundamental software compatibility constraints and by the computational demands of advanced robotic operation.

The CMS and LIX robotics framework is built on ROS2, MoveIt2, and advanced computer vision pipelines. ROS2 is natively supported only on Ubuntu and is not compatible with RedHat. While virtual machines and container-based workarounds are currently possible, they introduced additional complexity, limit accessibility for beamline staff, and reduce long-term sustainability. A native Ubuntu system is therefore required for reliable development, debugging, and routine operation.

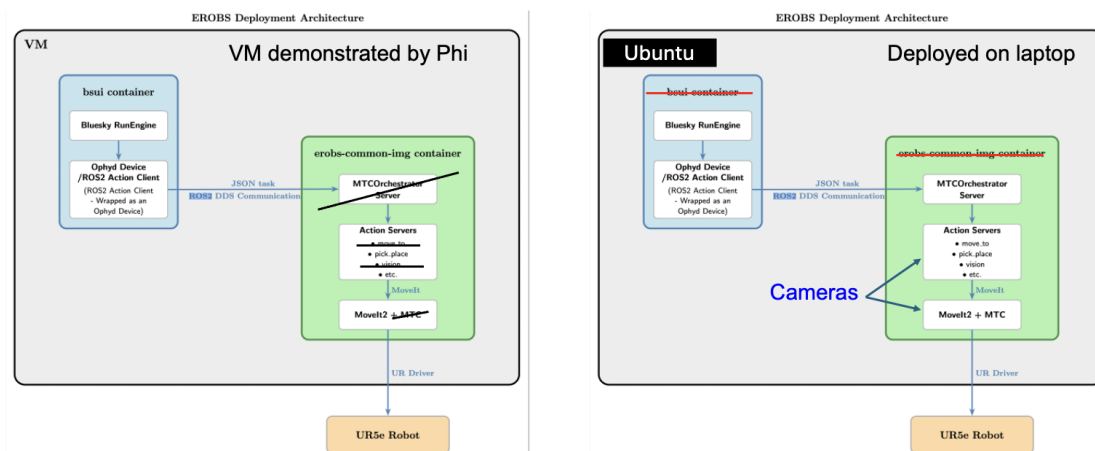
In addition, NVIDIA Isaac Sim, which is used to create high-fidelity digital twins of the robots, tools, and beamline environment, requires both Ubuntu and an NVIDIA GPU to run. Isaac Sim enables safe testing of motion planning, collision avoidance, camera placement, and experimental workflows without risking physical hardware. These simulations are also essential for validating autonomous behaviors and for developing and testing AI/ML-enabled capabilities.

The absence of an Ubuntu system with an NVIDIA GPU has a direct impact on the continue development of this robotic platform:

- Real-time vision, point-cloud processing, and safety monitoring cannot be performed.
- Digital-twin based validation of experiments
- Autonomous robotic experiments are severely hindered or infeasible.
- Commissioning time during beamtime increases, reducing experimental efficiency and increasing operational risk.
- Certain experiments that rely on robotic handling cannot be performed at all, particularly those requiring multiple steps to prepare the sample for *insitu* experiments.

For these reasons, the Ubuntu computer with an NVIDIA GPU is a critical enabling component of the CMS and LIX robotic system. Without it, key robotic capabilities cannot be deployed, and the scientific scope enabled by robotic automation would be significantly reduced.

Challenges:



Comparison of two structures to operate robot by bsui. Left: VM structure demonstrated by Phil. Right: Ubuntu structure deployed on laptop

1. Compatibility of ROS2.
 ROS2 operation relies on Ubuntu 22, which is not compatible with Redhat, the standard OS in NSLS II. The current solution is to deploy two separate containers for ROS2 and bsui within a Virtual Machine. This approach makes it difficult for general users and even beamline staff to access or modify the system. A more friendly strategy will be needed for long-term sustainability.
2. Communication between ROS2 and bluesky.
 Currently, the communication is handled by ROS between the containers inside the VM. This approach works but is not well integrated with the broader beamline control and has latency/network issues when using a VM. A more robust solution would involve using a Ubuntu workstation to facilitate reliable communication between Bluesky and ROS and creating:
 - a ROS2 node capable of publishing and subscribing to Ophyd devices.
3. Coordination of cameras.
 The safety operation of the robot relies on point clouds generated from three cameras: two USB stereo cameras permanently mounted on the hutch walls, and one high-resolution Ethernet stereo camera mounted on the robot's end effector. The fixed cameras provide global spatial awareness over the open area, while the arm-mounted camera enables fine-resolution (<1 mm). Effective calibration and synchronization are essential for safe and precise robotic operation.
4. GPU resources for safety operation, including monitoring the points of cloud and performing digital twins' simulation is required.

Technical Specifications for GPUs for the Cameras